

Documentオブジェクト

前回、**window** オブジェクト はブラウザ全体の「社長さん」だと説明しましたね。今回の **document** オブジェクト は、**window** オブジェクトの下にあるプロパティで、その社長さんの下で働く「現場監督（げんばかんとく）」です。

[!IMPORTANT]

オブジェクトは別のオブジェクトのプロパティになることがあります。

学生の方は、「ウェブページという建物を管理し、中身を書き換えたり、新しくパーツを作ったりする責任者」と考えると、イメージが湧きやすくなるかもしれません。

documentオブジェクトとは、ブラウザに表示されている**HTML（ウェブページの中身）** そのものを指します。

JavaScriptを使って、文字の色を変えたり、ボタンが押されたときに動きをつけたりするのは、すべてこの「現場監督（document）」に命令を出しているからです。

これを専門用語で **DOM（ドム / Document Object Model）** と呼びます。HTMLを「木の枝」のような構造として管理しているのが特徴です。

DOMツリーについては、次のセクションで、くわしく説明します。ここでは、Windowとdocumentの違いをしっかりと頭に入れてください。

1. documentオブジェクトとは？

ブラウザに表示されている**HTML（ウェブページの中身）** そのものを指します。

JavaScriptを使って、文字の色を変えたり、ボタンが押されたときに動きをつけたりするのは、すべてこの「現場監督（document）」に命令を出しているからです。

これを専門用語で **DOM（ドム / Document Object Model）** と呼びます。HTMLを「木の枝」のような構造として管理しているのが特徴です。

2. よく使うプロパティ（ページの情報）

現場監督が知っている、今のページの情報は、

- **document.title**

ブラウザのタブに表示されている「ページのタイトル」です。JavaScriptで書き換えることもできます。

- **document.body**

HTMLの **<body>** タグの中身全体です。ページ全体の背景色を変えたいときなどに使います。

- **document.URL** 現在開いているページの住所（URL）を教えてください。

3. よく使うメソッド（現場監督への命令）

これが一番大切です！ 特定の場所を探したり、中身を変えたりする命令です。

場所を探す（セレクト）

- `getElementById("ID名")` 特定のIDがついたパーツを1つだけ探します。一番速くて正確です。
- `querySelector("セレクト")`
CSSと同じ書き方（`.class` や `#id`）でパーツを探せます。とても便利です！

これらは、以前にも出てきましたね。実はdocumentオブジェクトのメソッドだったのです！

[!note]

`document.getElementById("ID名")` のように書いてもかまいませんが、普通は `document.` を省略して、`getElementById("ID名")` と書きます。

中身を書き換える・作る

- `createElement("タグ名")` 新しいパーツ（`<div>` や `<li>` など）をメモリの中に作ります。
- `write("文字")`
ページに直接文字を書き込みます（※あまり使いませんが、基本として覚えておきましょう）。

4. 実際に動かしてみよう！

Visual Studio Code を使って、以下のようなhtmlを作成しましょう。名前は `doc_obj.html` としましょうか。

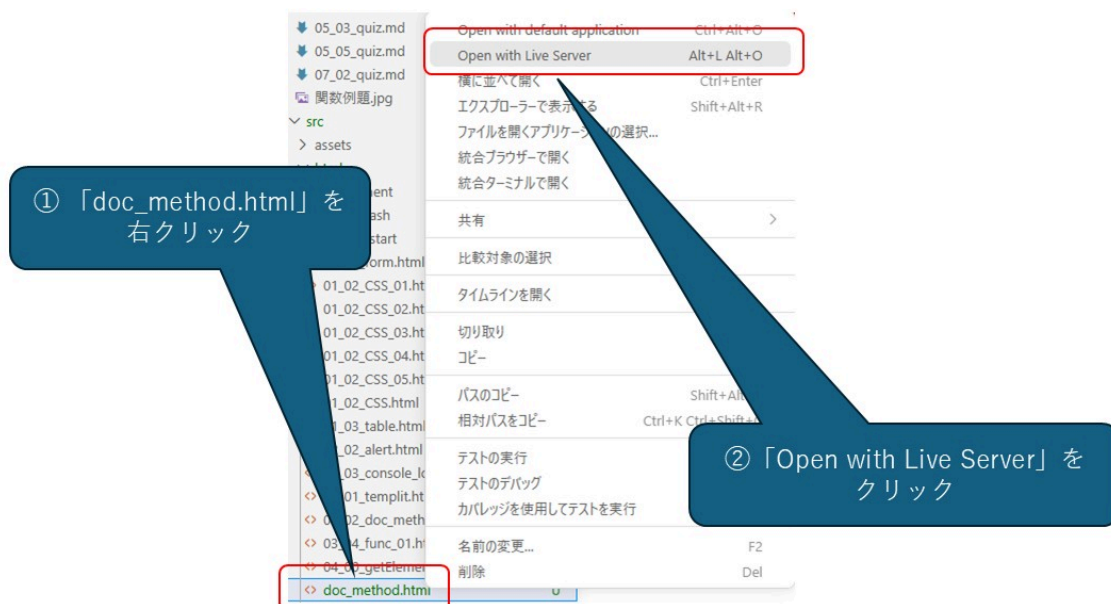
```
<!DOCTYPE html>
<html lang="ja">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
  </head>
  <body>
    <h1 id="greeting">こんにちは！</h1>

    <p>コンソールを開いて、下のJavaScriptを入力してみましょう。</p>

    <pre>
      // 1. IDが "greeting" というパーツを探す
      const message = document.getElementById("greeting");
      // 2. その中身を書き換える
      message.textContent = "こんにちは、日本へようこそ！";
      // 3. スタイル（色）も変えちゃう
      message.style.color = "red";
    </pre>
```

```
<a href="javascript:history.back();">戻る</a>
</body>
</html>
```

保存できたら、Visual Studio Code 上から、Open with Live Severを使って、開きましょう。



ブラウザに表示された指示に従って、コンソールにJavaScriptを入力しましょう。

コンソールを開くには、F12 キー（ノートPCは Fn + F12） または Ctrl + Shift + I を使います。

オブジェクトと const の意外な関係

上のコードを見て、疑問に思いませんか？「const で宣言された変数は、後から値を変更できないはずでは？」 そう思った人は、変数の説明をととてもよく覚えている人です。

しかし、上のコードはエラーになりません。それについて、説明しましょう。

JavaScriptの初心者が混乱しやすい点のひとつが、オブジェクトと const の意外な関係です。

「const で宣言した変数は中身を変えられない」と説明しましたが、オブジェクトの場合は少し違います。

- 名札の固定（参照の不変性）：const で作ったオブジェクトそのものを別のオブジェクトに入れ替えることはできません（別の箱に名札を付け替えるのは禁止）。
- 中身の変更はOK：箱の中に入っている個別のデータ（プロパティ）を書き換えることは可能です。

「強力なガムテープで名札が貼られた箱」をイメージしてください。ガムテープで固定されているので、名札を別の新しい箱に貼り直すことはできません。でも、箱のふたをひとつ開けて、中に入っているお菓子

を入れ替^{い か}えたり、ジャムを塗^ぬったりすることは自由^{じゆう}なのです。

たと^{たと}え^えるなら、「ファイルの保存場所^{ほぞんばしょ}（パス）」は固定^{こてい}されているけれど、「ファイルの中身^{なかみ}」は自由^{じゆう}に書き換^かえられるようなイメージです。

先^{さき}ほどの例^{れい}では、`const`を使^{つか}って宣言^{せんげん}した`message`という箱^{はこ}を開^あけて、中身^{なかみ}を"こんにちは、日本^{にほん}へようこそ！"に入れ替^{い か}えました。これはエラーにはなりません。

しかし、

```
// 1. IDが "greeting" というパーツを探す
const message = document.getElementById("greeting");
// 2. タグがpのパーツを探す
message = document.querySelector("p")
```

はエラーになります。`message`というラベルを「"greeting"というIDの箱^{はこ}」からはがして、「"p"というタグの別^{べつ}の箱^{はこ}」に貼^はろうとしているからです。

`const`は`message`というラベルと、それを入^いれる入^いれ物^{もの}との関係^{かんけい}を固定^{こてい}して、ラベルの張替^{はりか}えを許^{ゆる}しません。

しかし、あくまでラベルと入^いれ物^{もの}との関係^{かんけい}であって、入^いれ物^{もの}が同じ^{おな}であれば、その中身^{なかみ}が変^かわっても、かま^かわないのです。

5. 学生^{がくせい}へのアドバイス：`window`と`document`の使^{つか}い分^わけ

「どっちを使^{つか}えばいいの？」と聞^きかれたら、こ^こう答^{こた}えてあげてください。

「ブラウザの機能^{きのう}（タイマー、アラート、URL移動^{いどう}）を使^{つか}いたいときは`window`」

「ページの中身^{なかみ}（文字^{もじ}、画像^{がぞう}、ボタン）をいじりたいときは`document`」

まとめテーブル

めいれい じょうほう なまえ 命令・情報 ^{めいれい} の名 ^な 前 ^{まえ}	やくわり 役割 ^{やくわり}	たと ばなし 例 ^{たと} え話 ^{ばなし}
<code>title</code>	ページのタイトル	ほん ひょうし なまえ 本の表紙 ^{ほん} の名 ^な 前 ^{まえ}
<code>getElementById()</code>	とくてい しめい 特定のパーツ ^{とくてい} を指 ^{しめい} 名 ^{めい} する	しゅっせきばんごう ばん ひと よ 「出席 ^{しゅっせき} 番 ^{ばん} 号 ^{ごう} 〇番 ^{ばん} の人 ^{ひと} ！」と呼 ^よ ぶ
<code>querySelector()</code>	じゆう さが 自由 ^{じゆう} にパーツ ^{さが} を探 ^{さが} す	あか ふく き ひと さが 「赤 ^{あか} い服 ^{ふく} を着 ^き てい ^{ひと} る人 ^{さが} ！」と探 ^{さが} す
<code>textContent</code>	もじ か か 文字 ^{もじ} を書 ^か き換 ^か える	かんばん もじ か なお 看 ^{かん} 板 ^{ばん} の文 ^{もじ} 字 ^か を書 ^か き直 ^{なお} す
<code>createElement()</code>	あた ^{あた} ら ようそ つく 新 ^{あた} しい要 ^{よう} 素 ^そ を作 ^{つく} る	あた ^{あた} ら ようい 新 ^{あた} しいレンガ ^{ようい} を ^{レンガ} 用 ^{よう} 意 ^い する

いかがでしょうか？「現場監督^{げんぱかんとく}の document くん」がいれば、ウェブページを思った^{おも}通り^{とお}に操^{あやつ}れるようになります。